

Parquet vs CSV on Object Storage

Storage, Transfer, and Query Trade-offs

Salsabil Mtiraoui Erulan Ibraimov

Cloud Computing and Big Data — University of Neuchâtel, Spring 2026

<https://github.com/SalsabilMtiraoui/CCBD-variant1>

1 Introduction

CSV is universal: every tool can open it and any developer can read it. Apache Parquet is the opposite — binary, columnar, and opaque to humans — but designed from the ground up for analytical workloads on distributed object storage. This project benchmarks both formats on a synthetic financial transaction dataset, measuring storage footprint, transfer speed, and query latency across three backends: a local **MinIO** instance (Docker), **Azure Blob Storage**, and **AWS S3**.

Team contributions

Member	Contribution
Erulan Ibraimov	Data generation and benchmarking for MinIO and AWS S3
Salsabil Mtiraoui	First data generation run and benchmarking for Azure Blob Storage
Both	All remaining code, analysis, and report — reviewed by one after the other

2 Dataset and Generation

We generated synthetic financial transactions covering April 2026. The schema reflects a semi-realistic payment processor: four of the seven columns are low-cardinality categoricals (**region**, **event_type**, **currency**, **status**), which map directly to Parquet dictionary encoding — the main driver of the compression advantage measured throughout this report.

Table 1: Dataset schema.

Column	Type	Description
ts	timestamp	Transaction timestamp, 2026-04-01 to 2026-04-30
user_id	int64	User identifier, 1 to 1,000,000
region	cat. (5)	Eurozone 35%, US 35%, UK 15%, Canada 10%, Switzerland 5%
event_type	cat. (4)	payment 45%, transfer 20%, deposit 18%, withdrawal 17%
value	float64	Log-normal amount per event type (median \$26–\$257)
currency	cat. (5)	Native currency per region; 10% use a foreign currency
status	cat. (3)	completed ~93–96%, pending, failed

Generation uses fully vectorised NumPy operations: all columns are drawn simultaneously from pre-computed probability arrays with no per-row Python loops. Transaction values follow log-normal distributions per event type, which is standard for financial amounts — most are small but occasional large ones create the right-skewed tail typical of real payment data.

The dataset is split into fixed chunks of 5 M rows, producing three sizes S, M and L. The chunked layout keeps individual files at a manageable size, makes throughput measurements comparable across sizes via weighted averages over identical units. The multiple files in a dataset is usual in data lake ingestion practice.

Table 2: Dataset sizes. Parquet uses Snappy compression.

Label	Rows	Files	CSV size	Parquet size	Saving
S	5 M	1	299 MB	79 MB	73.6%
M	25 M	5	1,494 MB	395 MB	73.6%
L	100 M	20	5,976 MB	1,578 MB	73.6%

The compression ratio is a flat **73.6% at every scale** — not a coincidence. It is entirely schema-driven: with only 3–5 unique values per categorical column, Parquet’s dictionary encoding stores the string "Eurozone" once and references it by a small integer per row, rather than repeating it as raw UTF-8 text tens of millions of times as CSV does. This structural advantage does not depend on dataset size, which is why the ratio is identical across S, M, and L.

3 Methodology

Stack and backends

The pipeline (`dataset_gen.py` → `upload.py` → `download.py` → `bench.py`) uses **boto3** for S3-compatible storage, **azure-storage-blob** for Azure, and **pyarrow.dataset** as the unified query engine for both formats. Results are appended to `results.csv` after each run and aggregated in `analysis.ipynb`, where multiple runs are averaged where available.

MinIO was the primary benchmark environment: running locally via Docker, it uses the full S3 API identically to AWS but eliminates WAN latency entirely, providing a stable and reproducible baseline for format-level differences. Azure and AWS runs then expose the additional effect of real network conditions. All seeds are fixed at 42 for full reproducibility.

Aggregation and query design

Because all chunks have the same size (5 M rows, ≈ 300 MB CSV / ≈ 80 MB Parquet), the data is homogeneous by construction — no meaningful heterogeneity between chunks. Therefore, throughput could be computed as a **weighted average**: total MB divided by total seconds across all sizes. Different sizes simply produce proportionally more time; there is no interaction effect that would invalidate pooling them.

We ran **two region filters** on the same time window (April 10–20, 33% of the month) to probe the effect of filter selectivity. Both queries compute `COUNT(value)` and `MEAN(value)` `GROUP BY event_type` and run directly on all endpoints.:

- **Eurozone** — 35% of rows $\times$ 33% time $\approx$ **11.7% selectivity** (584 k rows in size S)
- **Switzerland** — 5% of rows $\times$ 33% time $\approx$ **1.7% selectivity** (83 k rows in size S)

Transfer throughput

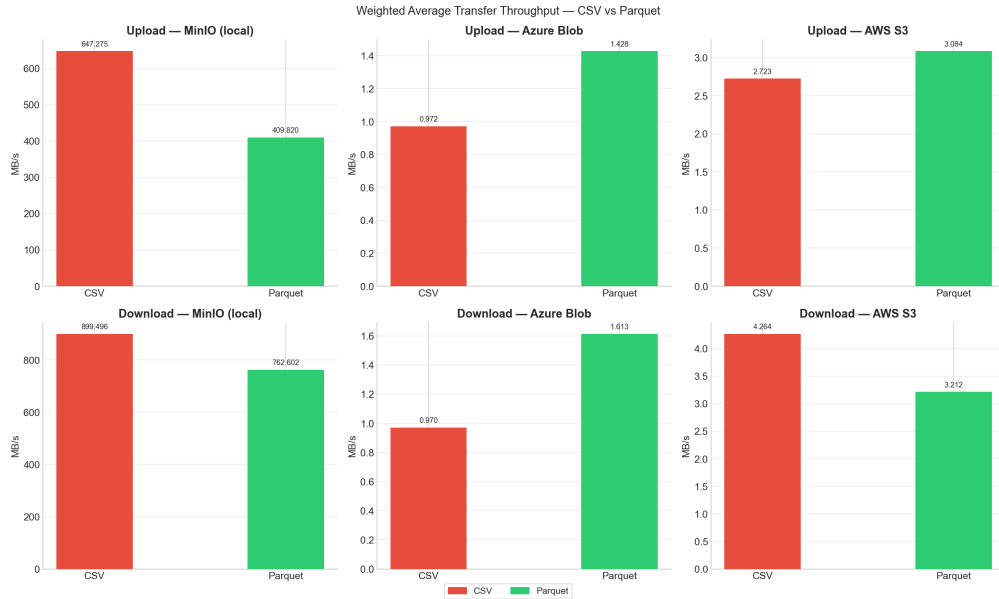


Figure 1: Weighted-average transfer throughput (MB/s) for upload and download, per backend. Note the very different y-axis scales between MinIO and the cloud backends.

Figure 1 shows the weighted-average throughput. The key insight is that Parquet is **not** intrinsically faster per byte — on MinIO, CSV actually achieves higher raw throughput (647 vs 410 MB/s upload) because the local disk handles both formats easily. What Parquet does is move **73.6% fewer bytes**. The net effect on total transfer time is a 2.6–6.8 $\times$ reduction depending on backend and size. On Azure, where WAN bandwidth is the bottleneck (≈ 1 MB/s), the size difference becomes decisive: a size-L CSV upload takes roughly 1h51 versus 18 minutes for Parquet. The speedup is slightly below the raw 3.79 $\times$ size ratio on MinIO because per-file connection overhead becomes proportionally larger at small file counts.

4 Object Listing

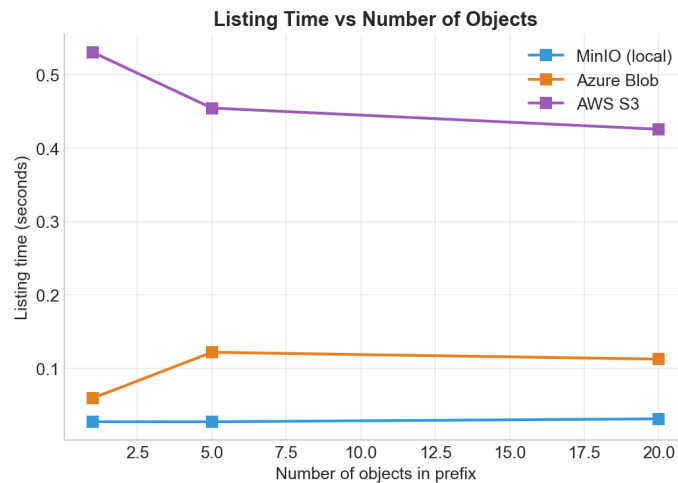


Figure 2: Listing time (seconds) vs number of objects per backend.

Table 3: Mean listing time (seconds) by backend and dataset size. CSV and Parquet have identical file counts per size, so listing is entirely format-agnostic.

Backend	1 file (S)	5 files (M)	20 files (L)
MinIO (local)	0.027 s	0.027 s	0.031 s
Azure Blob	0.059 s	0.122 s	0.113 s
AWS S3	0.531 s	0.455 s	0.426 s

Listing is format-agnostic since CSV and Parquet produce the same number of files per size. MinIO completes in under 35 ms at all sizes; Azure stays below 120 ms; AWS peaks at 530 ms at size S due to cross-region API latency rather than object count.

We collected multiple listing calls per configuration. Because the absolute values are so small (all under 600 ms), a single slow API response can noticeably shift the mean. The trend in Figure 2 shows the expected gentle increase with object count on Azure, while AWS actually decreases slightly — likely a caching effect on subsequent calls within the same session.

5 Analytical Query Performance

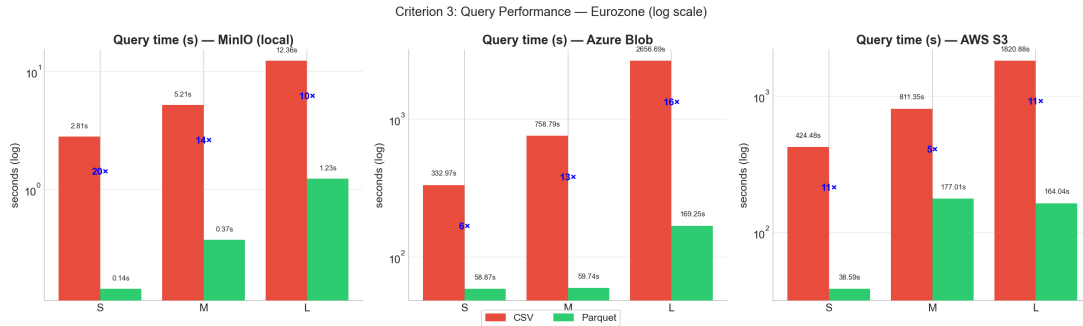


Figure 3: Query time (log scale) — Eurozone filter ($\approx 11.7\%$ selectivity). Blue labels show Parquet speedup over CSV per backend and size.

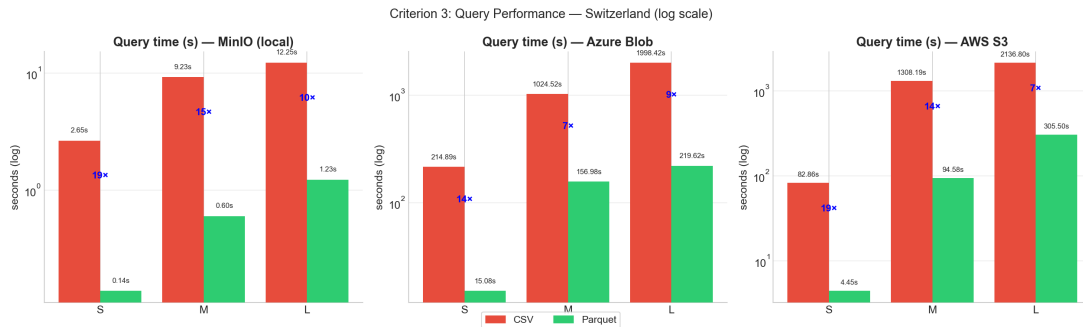


Figure 4: Query time (log scale) — Switzerland filter ($\approx 1.7\%$ selectivity). Despite returning $7\times$ fewer rows, speedups are nearly identical to Eurozone.

Figures 3 and 4 show the most striking result of the benchmark: a **5–20 $\times$ query speedup** for Parquet, consistent across all backends and dataset sizes.

Two PyArrow mechanisms drive this. **Column pruning:** the query only needs `event_type` and `value`; Parquet reads 2 of 7 columns and skips the rest at the storage layer. **Predicate pushdown:** Parquet stores per-column min/max statistics per row group — row groups where the `region` range cannot match the filter are skipped without reading any data. CSV has neither mechanism; every byte must be read and decoded before filtering can begin.

Why doesn't the narrower Switzerland filter give a much bigger speedup? Intuitively, filtering 1.7% of rows instead of 11.7% should be much faster. In practice the query times are nearly identical between the two regions. For CSV, the cost is reading all 7 columns for all rows — selectivity is irrelevant because all data must be scanned anyway. For Parquet, the cost is dominated by streaming the 2 pruned columns — column pruning already eliminates 5/7 of the I/O, and further row-group skipping by region provides limited extra benefit because region values are uniformly distributed across row groups with no clustering. The speedup is structural, not selectivity-driven.

The speedup decreasing with size on MinIO (20× at S to 10× at L) reflects growing per-file metadata overhead: size L opens 20 Parquet files, and each file's footer read accumulates. On Azure and AWS this effect is masked by network I/O dominating total time.

6 Cost Estimation

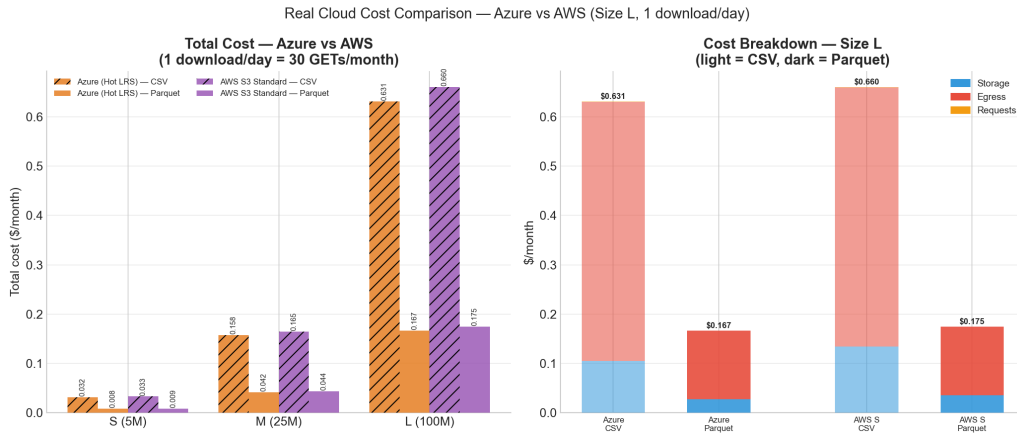


Figure 5: Real cloud cost comparison (Azure Hot LRS West Europe vs AWS S3 eu-central-1). Left: total monthly cost per size and format. Right: cost breakdown for size L — egress dominates at approximately 83% of total.

The saving is a consistent **73.5% on both providers at every size**. The stacked breakdown for size L in Figure 5 makes the mechanism clear: egress ($\approx 83\%$ of total) is priced at 4–5× the monthly storage rate on both providers. Since Parquet reduces file size by 73.6%, it reduces egress by the same proportion, which drives almost the entire cost reduction. Storage fees follow the same ratio; request costs (PUT/GET on 1–20 objects) are negligible in comparison. At size L on AWS, this translates to roughly \$0.48/month saved per daily access cycle — modest in isolation, but scaling linearly with access frequency and dataset count in production.

7 Interpretation and Recommendations

The results across all three backends and all three dataset sizes point to a consistent conclusion: Parquet outperforms CSV on every dimension measured. The advantage is not marginal — a 73.6% storage reduction, 5–20 $\times$ query speedup, and 73.5% cost saving are transformative at production scale.

Use Parquet for any analytical workload on object storage: gains are consistent on MinIO, Azure, and AWS, and the 73.6% size reduction translates almost directly to a 73.5% cost reduction since egress dominates cloud billing. Target files of 80 MB–1 GB (our 5 M-row chunks land naturally in this range). For production data lakes, add Hive-style partitioning (`region=Eurozone/date=2026-04-10/`) to unlock directory-level pruning beyond what we measured.

Use CSV for human-readable exchange with legacy systems, small one-off files written once and rarely queried, and quick debugging with shell tools.

Limitations

Single runs for large sizes: L-size cloud benchmarks were run once, so measurements cannot be separated from network noise (e.g. the anomalously low AWS size-M speedup of 2.1 $\times$ vs 4–5 $\times$ elsewhere).

Synthetic uniform data: real financial data has temporal clustering and regional skew that would affect compression ratios and row-group skipping differently.

No partitioning tested: adding `region/date` partitions is the most impactful next step and would likely widen query speedups for narrow filters.

One codec and query shape: queries requiring all 7 columns would reduce Parquet’s column-pruning advantage.

Conclusion

Table 4: Summary of results across all backends and dataset sizes.

Dimension	Parquet result	Key driver
Storage size	73.6% smaller at every scale	Dictionary encoding of categoricals
Generation speed	5.5 $\times$ faster per chunk	Binary write vs text serialisation
Transfer time	2.6–6.8 $\times$ faster	Fewer bytes to move
Query speed	5–20 $\times$ faster	Column pruning + predicate pushdown
Cloud cost	$\approx$ 73.5% cheaper	Egress savings dominate

Across 5 M, 25 M, and 100 M financial transactions on three storage backends, Parquet consistently outperforms CSV on every metric that matters for analytical pipelines (Table 4). CSV remains the right choice for human-readable exchange and legacy interoperability — for everything else on object storage, the numbers speak unambiguously in favour of the columnar format.